Understanding Framer: Purpose, Function, Adoption

https://gemini.google.com/app/622c11ff49868910

User prompt: Can you tell me what is Framer? it's purpose? how it works ? and what is adoption of it in community. let me know like I heard this for the First time : https://www.framer.com/

Response: If you are hearing about Framer for the first time, the easiest way to understand it is to think of it as **"Figma, but it creates actual, functioning websites instead of just static pictures."**

1. What is Framer?

Framer is a modern, cloud-based website builder designed primarily for UI/UX designers, marketers, and startups. It bridges the massive gap that has traditionally existed between *designing* a website and *coding* it.

Historically, a designer would draw a website in a tool like Figma, hand the image files over to a software engineer, and the engineer would write the code to make it real. Framer skips the engineering step entirely. You design the site visually, click "Publish," and it is instantly a live, high-performance website on the internet.

2. What is its Purpose?

The main purpose of Framer is to give creative control back to design and marketing teams, allowing them to ship professional websites at lightning speed without being bottlenecked by developers.

It is specifically optimized for:

- **High-End Marketing Sites:** Landing pages, product launches, and brand experiences.
- **Immersive Visuals:** Sites that rely heavily on jaw-dropping animations, smooth page transitions, and complex scroll effects.
- **Speed to Market:** Turning an idea into a beautifully designed, live MVP (Minimum Viable Product) or portfolio in hours rather than weeks.

3. How Does It Work?

If you've ever used a design tool like Figma or Sketch, Framer will feel completely natural.

- **The Canvas Interface:** You work on an infinite digital canvas where you drag-and-drop text, shapes, buttons, and images.
- **Figma Copy-Paste:** Framer has a direct integration with Figma. You can literally highlight a design you made in Figma, hit `Ctrl+C`, go into Framer, hit `Ctrl+V`, and it imports the layout into the web-builder.
- **No-Code Animations:** Framer is powered by an animation engine called *Framer Motion*. Instead of writing complex JavaScript code for animations, you just click an object and select options like "Fade in on scroll" or "Spring react on hover" using a visual menu.
- **Built-in CMS (Content Management System):** If you want to run a blog, a job board, or a portfolio, Framer has a spreadsheet-like database built right in. You update the text in the database, and it automatically creates new pages using your visual templates.
- **AI Assistance:** Framer features advanced AI website generation capabilities. You can type a prompt like *"Build a dark-themed portfolio for a blockchain developer,"* and it will lay out the structure, write placeholder copy, and color-code the site for you to edit.

- **Instant Edge Hosting:** Framer hosts the website for you. When you click "Publish," it converts your visual design into optimized, highly-performant HTML, CSS, and React code, deploying it to a global content delivery network (CDN) so your site loads instantly worldwide.

4. What is the Adoption of Framer in the Community?

Framer has exploded in popularity, transitioning from a niche prototyping tool to a dominant player in the web design space.

- **The Startup and Y Combinator Choice:** Framer has become the go-to default tool for modern startups. Data shows that roughly **40% of startups in recent Y Combinator batches** launched their main company websites using Framer because of how fast it allows them to iterate.
- **The Designer Community Loyalty:** Because the interface mirrors Figma so closely, hundreds of thousands of UI/UX designers have adopted it to offer full web-development services to their clients without ever learning how to code.
- **Market Positioning:** While legacy tools like WordPress still dominate the raw numbers of the web (64%+ market share), Framer actively competes in the premium, modern "no-code" space alongside **Webflow**.
    - *Webflow* is generally preferred for massive corporate data structures, complex e-commerce, and heavy backend engineering.
    - *Framer* is rapidly stealing market share from competitors because it is significantly cheaper, faster to learn, and infinitely better at smooth animations out of the box.
- **Enterprise Adoption:** Major tech brands (like Mixpanel, Zapier, and standard-setting design agencies) use Framer for their marketing hubs, reporting that their design teams can push updates **3x faster** than when they relied on traditional engineering queues.

Summary Summary: If you want a website that looks incredibly beautiful, moves fluidly with animations, and can be built entirely visually without touching a line of code, Framer is currently one of the best tools in the world to do it.

Sources

1. Companies Using Framer in 2026: Full Customer List & Insights - TechnologyChecker.io
2. https://www.framer.com/solutions/website-builder/
3. What Is Framer? A Design Tool Guide for 2026 - UI Things
4. Framer for Enterprise Websites in 2026: CMS, Pricing & Scale Limits - Omakase Design
5. Framer vs Webflow 2026: Which Platform Wins? - Verlua
6. Framer vs Webflow: Expert Comparison & Verdict (2026) - Pixeto

---

User prompt: I was reding developer guide of Framer - where they mention 'Plugins' word. so what it means? https://www.framer.com/developers/plugins-introduction

Response: In the context of Framer’s developer guide, a **Plugin** is essentially a mini-application that lives inside the Framer editor to automate tasks, extend the tool's capabilities, or connect Framer to other external web services.

If you have used tools like VS Code, Chrome, or Figma, it’s the exact same concept as an "extension" or "add-on."

Here is a breakdown of what Plugins mean for developers in Framer, how they function, and what you can do with them:

1. What does a Plugin actually do?

According to Framer's developer documentation, a plugin can interact directly with the Framer user interface and canvas. It usually manifests as a floating window or runs invisible background tasks to:

- **Manipulate the Canvas (Nodes):** A plugin can programmatically add, delete, resize, or modify layers, shapes, and text frames on the visual design board.
- **Inject Code Components & Assets:** It can automatically drop custom React code components, SVGs, images, or UI kits right onto a user's page.
- **Sync with the CMS (Content Management System):** A plugin can fetch data from an external database (like Airtable, Shopify, or a custom backend) and automatically push or sync that content directly into Framer's built-in CMS collections.
- **Inspect Site Data:** It can read information about the current Framer project, such as viewing details about the published site.

2. How are they built? (The Tech Stack)

For developers, Framer plugins are built using standard web technologies: **HTML, CSS, and JavaScript/TypeScript**.

Under the hood, a plugin runs inside a secure `iframe` within the Framer application. Framer provides a specialized **Plugin API** (a set of pre-written functions) that allows your mini-app's JavaScript code to "talk" to the Framer canvas.

- *Want a button in your plugin window that creates 50 text layers?* Your HTML button triggers a JavaScript function using Framer's API (like `framer.createNode()`), and Framer draws it instantly.

3. Practical Examples of Plugins

To make it real, here are scenarios where a developer would build or use a plugin:

- **Data Importer:** A plugin that connects to a company's real estate database. Instead of a designer copying and pasting 100 house listings manually, the plugin pulls the data via API and auto-generates the pages.
- **Design Helpers:** Plugins that check the canvas for accessibility (e.g., ensuring text contrast is high enough for visually impaired users) or clean up messy, unnamed layers.
- **AI Add-ons:** A plugin that uses the OpenAI API to rewrite highlighted text into different tones (e.g., "make this paragraph sound more professional").
- **Asset Management:** Connecting Framer directly to a company's cloud storage (like Google Drive or Box) so designers can pull approved brand logos directly into their layout.

Why this matters to you as a Developer

If you are learning Framer from a developer's standpoint, plugins mean you aren't restricted by the built-in UI. If Framer doesn't natively support a feature you or your team needs, you can write a standard web app, package it as a plugin, plug it into the Framer API, and completely customize the creation environment.

User prompt: so at this point, Here is the image of 'Framer' in my mind : Framer is like a UI-UX editor like Figma where you can simpliy build your web design. but one more step ahead - we can directly create our website from here without providing design to developer. [I don't know how but I understand it's purpose] - Framer provide a canvas where it maintains the layerrs and components and Flexible to handle good animation and UX. Now we can talk with Framer using plugin. but still I get confuse - let's say I have website created in React - so do I need to use Framer with my existing code and component

of react - if yes then how and for why? or our whole website should build in Framer itself? can you make it clear for me?

Response: Your mental image of Framer is **100% accurate**! You have understood its core purpose perfectly.

To clear up your confusion about how it interacts with an existing React website, you have touched on one of the most important architectural decisions teams face. The short answer is: **Usually, your whole marketing website is built inside Framer itself, but Framer provides powerful bridges to bring your custom React code into it, or vice versa.**

Let's break down the two main ways you can handle this, depending on what your React app actually does.

Scenario A: Your whole website/landing page is built inside Framer (The Most Common Way)

If you are building a landing page, a marketing site, a portfolio, or a blog, **you should build the entire thing inside Framer.**

You do not write a separate React app for it. You use Framer's visual canvas, and when you hit "Publish," Framer handles all the React generation and hosting under the hood. You don't touch code, and you don't maintain a repository.

Scenario B: You already have a React Application (e.g., a SaaS Dashboard, a Web App)

Let's say you have a web application like a crypto dashboard, an analytics tool, or a software portal built in React (using Next.js or Vite). You **cannot** build a complex, data-heavy dashboard completely inside Framer because Framer is meant for visual design, not complex backend logic, authentication, or state management.

In this case, you blend the two worlds together using one of two methods:

Method 1: Split the Architecture (Highly Recommended)

This is what 90% of tech companies do. They split their website into two different domains:

1. **The Marketing Site (Built entirely in Framer):** Hosted at `[www.mywebsite.com](https://www.mywebsite.com)`. The marketing team designs and updates this easily in Framer without touching code.
2. **The Actual Product App (Built in React/Next.js):** Hosted at `app.mywebsite.com`. When a user clicks "Log In" or "Sign Up" on the Framer site, they are redirected to your custom React app.

Method 2: Bringing custom React components *into* Framer

What if you want your Framer landing page to have a highly specialized interactive feature that you *already* coded in React? (For example, a live token-swapping widget, an interactive interactive calculator, or a live data chart).

Framer allows you to write or import **Code Components** directly into its editor.

- **How it works:** Inside Framer, you can open a built-in code editor and paste your custom React component code (using standard React, hooks, and props).
- **Why do this?** Framer will compile your code and turn your React component into a visual widget. A designer can then drag-and-drop your custom React widget onto the Framer canvas, resize it visually, and place it right next to normal text and images. You can even expose React `props` as

visual controls in Framer's right-hand sidebar so designers can change colors or text without touching your code.

Method 3: Using Framer Motion in your standalone React App

Framer also maintains a famous, open-source React animation library called **Framer Motion**. If you decide you don't want to use the Framer website builder at all, but you *love* the smooth, fluid animations Framer is famous for, you can simply install the library into your existing React project:

```
npm install framer-motion
```

This lets you use Framer's production-ready animation engine entirely via code inside your own standalone React application.

Summary Checklist for you:

- **Building a marketing site, landing page, or portfolio?** Build it 100% in Framer. No separate React code needed.
- **Building a complex software web-app with databases and user logins?** Write it in your own React codebase, but use Framer to build the public-facing marketing pages on a sub-domain.
- **Need a complex, custom feature inside a Framer page?** Code that specific piece in React and import it into Framer as a "Code Component."

User prompt: Great! so can you simplify what this person want to build and how with framer? I want a simple way to understand it : Build it as a code component. Not with Framer's Fetch. Fetch can't loop through arrays, so you can't build a grid with it. Write a React code component and do the fetching inside it. Handle what happens when things go wrong. We're telling you upfront: this API fails on purpose. Roughly 1 in 3 requests returns a 404 or 500. Both endpoints. That's not a bug, that's the test. Four situations. Loading. Error. Zero results. Working. If your page goes blank or dumps a raw error on screen, you lose this section. And think about what happens when the country call fails but the course call works. What do you show? There's no single right answer. There are wrong ones. Only GET works. Every other method returns a 405. If your component is sending anything else, ask yourself why. Give us two property controls. Someone who can't code should be able to change something from the Framer panel without touching your code. You pick which two. Pick the ones a designer would actually ask for. Make it work on phones. 3 columns on desktop. 2 on tablet. 1 on mobile. Nothing should break in between. Remember the card count varies, so the grid can't assume a nice round number. Don't hardcode the data. Here is the Requirement : What to build A landing page in Framer for a fake learning platform. Call it Skillpath. One section of that page pulls live data from an API. That section is what we're actually looking at. The rest is just the stuff around it. The page needs three things. A hero. Headline, one line under it, one button. Design it however you want. A courses section. This is the real test. More on it below. A footer. Three links and a copyright line. Don't overthink it.

Response: Here is a simple, plain-English breakdown of exactly what this person is asking you to build, why they want you to build it this way, and how it translates to code.

Think of this as a practical developer assignment designed to test how you handle **unreliable API data** inside a visual tool like Framer.

1. The Big Picture: What are you building?

You are building a landing page for a fake education website called **Skillpath**. Most of the page is just standard, simple text and design (a Hero banner at the top, a tiny Footer at the bottom).

However, the **middle section** must display a grid of online courses. This middle section cannot be built visually using Framer's built-in "Fetch" tool. It **must be coded by hand as a custom React component** inside Framer.

2. Why do you need a custom React Component?

Framer has a basic visual tool called "Fetch" to grab data from the internet, but it has a limitation: it struggles to loop through lists (arrays) of data to dynamically generate a grid of visual cards.

Because you need to pull a list of courses from an API and map them into a grid, you need to use standard React code (`useState`, `useEffect`, `.map()`) to handle the data fetching and rendering.

3. The Core Challenge: The Unreliable API

The person explicitly warns you: **The API endpoint you are hitting is broken on purpose.** One out of every three times you open the page, the API will fail (returning a server error like 404 or 500).

Your React component must be smart enough to handle **4 distinct states** gracefully without crashing the website or leaving a blank white screen:

1. **Loading State:** Show a nice loading spinner or text while the data is fetching.
2. **Error State:** If the API fails (1-in-3 chance), show a friendly message like *"Oops! Something went wrong. Please refresh the page."*
3. **Zero Results State:** If the API works but returns an empty list `[]`, show a message like *"No courses available right now."*
4. **Working State:** If the API works perfectly, loop through the data and render the course cards cleanly.

Bonus Brain Teaser: The API fetches two things—Countries and Courses. If the country list fails but the courses work, how does your UI handle it? You need to design a fallback (e.g., hide the country filter, or show a default country) so the app stays functional.

4. Layout Rules (Responsive Grid)

You need to style the course cards using CSS (like CSS Grid or Flexbox) so that the layout adjusts automatically based on the user's screen size:

- **Desktop screens:** 3 columns of cards side-by-side.
- **Tablet screens:** 2 columns.
- **Mobile screens:** 1 column (cards stacked vertically).

The layout cannot rely on a fixed number of items (like assuming there will always be exactly 6 cards). It must adapt whether the API sends back 2 cards or 20 cards.

5. Framer Integration (Property Controls)

Even though you are writing this section in pure React code, a non-technical designer still needs to be able to tweak it from the Framer sidebar.

You must expose **two settings** to the Framer UI using `addPropertyControls`. You get to choose which two, but they should be things a designer cares about. Excellent choices would be:

- **Control 1:** A text input to change the section's heading title (e.g., changing "Explore Courses" to "Trending Classes").
- **Control 2:** A number input to limit the max number of courses displayed (e.g., showing only the top 3 or top 6 courses).
- *(Alternative)* A color picker to change the background color of the course cards.

Summary Checklist of your React Code:

- [ ] Use only `GET` requests to fetch the data.
- [ ] Write a standard React component with `useEffect` to trigger the fetch on load.
- [ ] Use `try / catch` blocks to capture the deliberate 404/500 API failures.
- [ ] Use state variables (`loading`, `error`, `courses`) to switch between the 4 UI situations.
- [ ] Use responsive CSS media queries for the 3-2-1 column grid.
- [ ] Export `addPropertyControls` at the bottom of the file so the Framer panel gets its two sliders/inputs.

User prompt: So here What I understand, tell me if I am wrong : - we have an 2 APIs : 1st for course metadata and 2rd for country conversion. - There are two layer of Frontend might be require : 1) React 2) Framer React should able to handle course component properly with all problem and state mention in requirement. Framer helps us to provide a flexible way to adjust UI for any user by their own. as simple as that. right?

Response: You have hit the nail completely on the head! **You understand this perfectly.**

Your breakdown is exactly how a professional engineer thinks about a hybrid architecture like this. To confirm your logic, let's quickly map out exactly how those pieces fit together:

1. The Data Layer (The 2 APIs)

- **API 1 (Courses):** Gives you the raw data list (e.g., Course Title, Price, Image, Description).
- **API 2 (Countries):** Gives you currency exchange or country-specific data so you can dynamically change prices or text based on location.
- *Your understanding is spot on:* You have to write defensive code to handle the fact that one API might crash while the other works fine.

2. The React Layer (The Logic & Engine)

- You write standard React code inside Framer's built-in code editor.
- Your React code acts as the **safety shield**. It runs the `fetch()`, handles the `try/catch` errors, tracks the `isLoading` state, loops through the arrays using `.map()`, and manages the CSS media queries for the responsive 3-2-1 column grid.

3. The Framer Layer (The Visual Wrapper)

- Framer acts as the visual home for your code.
- The non-technical designer uses Framer to visually build the beautiful Hero section and the Footer without code.
- Then, they take your **React Code Component**, drop it into the middle section, and use those **2 Property Controls** you provided in the sidebar to change settings visually (like editing the title text or limiting the card count).

What this looks like structurally in Framer

When you open Framer to build this, your project hierarchy will look as simple as this:

```
[ Framer Visual Canvas ]
 ├── Hero Section (Built visually with text & buttons)
 ├── 🧩 Your React Code Component (Handles APIs, Errors, Grid, and Sidebar Controls)
 └── Footer Section (Built visually with links & copyright text)
```

You have completely demystified the requirements. You are ready to build it!
